MemGPT: Towards LLMs as Operating Systems

Charles Packer [1] **Sarah Wooders** [1] **Kevin Lin** [1]
Vivian Fang [1] **Shishir G. Patil** [1] **Ion Stoica** [1] **Joseph E. Gonzalez** [1]

Abstract

Large language models (LLMs) have revolutionized AI, but are constrained by limited context windows, hindering their utility in tasks like extended conversations and document analysis. To enable using context beyond limited context windows, we propose *virtual context management*, a technique drawing inspiration from hierarchical memory systems in traditional operating systems which provide the illusion of an extended virtual memory via paging between physical memory and disk. Using this technique, we introduce MemGPT (MemoryGPT), a system that intelligently manages different storage tiers in order to effectively provide extended context within the LLM's limited context window. We evaluate our OS-inspired design in two domains where the limited context windows of modern LLMs severely handicaps their performance: document analysis, where MemGPT is able to analyze large documents that far exceed the underlying LLM's context window, and multi-session chat, where MemGPT can create conversational agents that remember, reflect, and evolve dynamically through long-term interactions with their users. We release MemGPT code and data for our experiments at https://research.memgpt.ai.

1. Introduction

In recent years, large language models (LLMs) and their underlying transformer architecture (Vaswani et al., 2017; Devlin et al., 2018; Brown et al., 2020; Ouyang et al., 2022) have become the cornerstone of conversational AI and have led to a wide array of consumer and enterprise applications. Despite these advances, the limited fixed-length context windows used by LLMs significantly hinders their applicability to long conversations or reasoning about long documents. For example, the most widely used open-source LLMs can only support a few dozen back-and-forth messages or reason about a short document before exceeding their maximum input length (Touvron et al., 2023).

Directly extending the context length of transformers incurs a quadratic increase in computational time and memory cost due to the transformer architecture's self-attention mechanism, making the design of new long-context architectures a pressing research challenge (Dai et al., 2019; Kitaev et al., 2020; Beltagy et al., 2020). While developing longer models is an active area of research (Dong et al., 2023), even if we could overcome the computational challenges of context scaling, recent research shows that long-context models struggle to utilize additional context effectively (Liu et al., 2023a). As consequence, given the considerable resources needed to train state-of-the-art LLMs and diminishing returns of context scaling, there is a critical need for alternative techniques to support long context.

In this paper, we study how to provide the illusion of an infinite context while continuing to use fixed-context models. Our approach borrows from the idea of virtual memory paging that was developed to enable applications to work on datasets that far exceed the available memory by paging data between main memory and disk. We leverage the recent progress in function calling abilities of LLM agents (Schick et al., 2023; Liu et al., 2023b) to design MemGPT, an OS-inspired LLM system for **virtual context management**. Using function calls, LLM agents can read and write to external data sources, modify their own context, and choose when to return responses to the user.

These capabilities allow LLMs to effective "page" in and out information between context windows (analogous to "main memory" in operating systems) and external storage, similar to hierarchical memory in traditional OSes. In addition, function calls can be leveraged to manage control flow between context management, response generation, and user interactions. This allows for an agent to choose to *iteratively* modify what is in its context for a single task, thereby more effectively utilizing its limited context.

In MemGPT, we treat context windows as a constrained memory resource, and design a memory hiearchy for LLMs analogous to memory tiers used in traditional OSes (Patterson et al., 1988). Applications in traditional OSes interact

[1]University of California, Berkeley. Correspondence to: Charles Packer <cpacker@berkeley.edu>.

arXiv:2310.08560v2 [cs.AI] 12 Feb 2024

MemGPT：迈向将大规模语言模型作为操作系统

查尔斯·帕克 [1]，**莎拉·伍德斯** [1]，**凯文·林** [1]，**薇薇安·方** [1]，**希希尔·G·帕蒂尔** [1]，**伊恩·斯托伊卡** [1]，**约瑟夫·E·冈萨雷斯** [1]

摘要

大型语言模型（LLMs）已彻底改变人工智能，但受限于有限的上下文窗口，限制了其在扩展对话和文档分析等任务中的应用。为了实现超出有限上下文窗口的上下文利用，我们提出了虚拟上下文管理技术，该技术借鉴了传统操作系统中的层次化存储系统，通过在物理存储和磁盘之间进行分页，营造出扩展的虚拟存储的错觉。利用此技术，我们引入了MemGPT（MemoryGPT），一种智能管理不同存储层级的系统，旨在在LLM有限的上下文窗口内有效提供扩展的上下文信息。我们在两个领域对基于操作系统的设计进行了评估，这两个领域中现代LLMs的有限上下文窗口严重制约其性能：文档分析中，MemGPT能够分析远超基础LLM上下文窗口的大型文档；多会话聊天中，MemGPT可以创建能够记忆、反思并在与用户的长期交互中动态演变的对话代理。我们在https://research.memgpt.ai发布了实验所用的MemGPT代码和数据。

1. 引言

近年来，大型语言模型（LLMs）及其基础的变换器架构（Vaswani 等，2017；Devlin 等，2018；Brown 等，2020；Ouyang 等，2022）已成为对话式人工智能的基石，并引领了广泛的消费者和企业应用。尽管取得了这些进展，LLMs所使用的有限固定长度上下文窗口极大地限制了其在长对话或长文档推理中的应用。例如，最广泛使用的开源大型语言模型（LLMs）只能支持几十次的来回交互或对短文档进行推理，超出其最大输入长度（Touvron 等，2023）。

直接扩展变换器的上下文长度会导致计算时间和内存成本呈二次增长，这是由于变换器架构中的自注意力机制所致，使得设计新的长上下文架构成为一个紧迫的研究挑战（Dai et al., 2019；Kitaev et al., 2020；Beltagy et al., 2020）。尽管开发更长模型是一个活跃的研究领域（Dong et al., 2023），即使我们能够克服上下文扩展的计算挑战，近期研究表明长上下文模型在有效利用额外上下文方面仍然存在困难（Liu et al., 2023a）。因此，鉴于训练最先进的大规模语言模型所需的巨大资源以及上下文扩展的收益递减，迫切需要采用替代技术以支持长上下文。

在本文中，我们研究了如何在继续使用固定上下文模型的同时，提供无限上下文的错觉。我们的方法借鉴了虚拟内存分页的思想，该技术旨在通过在主存和磁盘之间分页数据，使应用程序能够处理远超可用内存的数据集。我们利用近年来在大语言模型（LLM）代理的函数调用能力方面的最新进展（Schick 等，2023；Liu 等，2023b），设计了 MemGPT，一种受操作系统启发的用于**虚拟上下文管理**的LLM系统。通过函数调用，LLM 代理可以读取和写入外部数据源，修改自身的上下文，并选择何时向用户返回响应。

这些能力使得大型语言模型（LLMs）能够在上下文窗口（类比操作系统中的“主存”）与外部存储之间高效“分页”信息，类似于传统操作系统中的层次存储结构。此外，函数调用可以用来管理控制流，在上下文管理、响应生成和用户交互之间进行切换。这使得代理能够选择迭代地修改其上下文中的内容，以更有效地利用其有限的上下文信息。

在 MemGPT 中，我们将上下文窗口视为一种受限的内存资源，并为大型语言模型（LLMs）设计了类似于传统操作系统（OS）中使用的内存层级（Patterson 等，1988）。传统操作系统中的应用程序与之交互

加州大学伯克利分校。联系方式：Charles Packer <cpacker@berkeley.edu>。

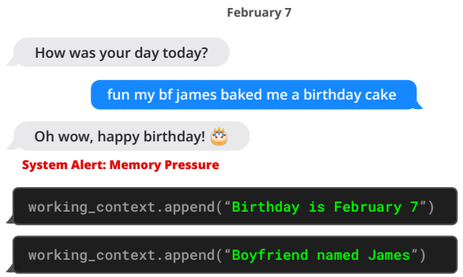

Figure 1. MemGPT (left) writes data to persistent memory after it receives a system alert about limited context space.

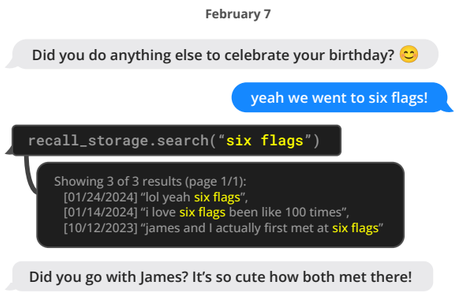

Figure 2. MemGPT (left) can search out-of-context data to bring relevant information into the current context window.

with *virtual memory*, which provides an illusion of there being more memory resources than are actually available in physical (i.e., main) memory by the OS paging overflow data to disk and retrieving data (via a page fault) back into memory when accessed by applications. To provide a similar illusion of longer context length (analogous to virtual memory), we allow the LLM to manage what is placed in its own context (analogous to physical memory) via an 'LLM OS', which we call MemGPT. MemGPT enables the LLM to retrieve relevant historical data missing from what is placed in-context, and also evict less relevant data from context and into external storage systems. Figure 3 illustrates the components of MemGPT.

The combined use of a memory-hierarchy, OS functions and event-based control flow allow MemGPT to handle unbounded context using LLMs that have finite context windows. To demonstrate the utility of our new OS-inspired LLM system, we evaluate MemGPT on two domains where the performance of existing LLMs is severely limited by finite context: document analysis, where the length of standard text files can quickly exceed the input capacity of modern LLMs, and conversational agents, where LLMs bound by limited conversation windows lack context awareness, persona consistency, and long-term memory during extended conversations. In both settings, MemGPT is able to overcome the limitations of finite context to outperform existing LLM-based approaches.

2. MemGPT (MemoryGPT)

MemGPT's OS-inspired multi-level memory architecture delineates between two primary memory types: **main context** (analogous to main memory/physical memory/RAM) and **external context** (analogous to disk memory/disk storage). Main context consists of the LLM *prompt tokens*—anything in main context is considered *in-context* and can be accessed by the LLM processor during inference. External context refers to any information that is held outside of the LLMs fixed context window. This *out-of-context* data must always be explicitly moved into main context in order for it to be passed to the LLM processor during inference. MemGPT provides function calls that the LLM processor to manage its own memory without any user intervention.

2.1. Main context (*prompt tokens*)

The prompt tokens in MemGPT are split into three contiguous sections: the **system instructions**, **working context**, and **FIFO Queue**. The system instructions are read-only (static) and contain information on the MemGPT control flow, the intended usage of the different memory levels, and instructions on how to use the MemGPT functions (e.g. how to retrieve out-of-context data). Working context is a fixed-size read/write block of unstructured text, writeable only via MemGPT function calls. In conversational settings, working context is intended to be used to store key facts, preferences, and other important information about the user and the persona the agent is adopting, allowing the agent to converse fluently with the user. The FIFO queue stores a rolling history of messages, including messages between the agent and user, as well as system messages (e.g. memory warnings) and function call inputs and outputs. The first index in the FIFO queue stores a system message containing a recursive summary of messages that have been evicted from the queue.

2.2. Queue Manager

The queue manager manages messages in **recall storage** and the **FIFO queue**. When a new message is received by the system, the queue manager appends the incoming messages to the FIFO queue, concatenates the prompt tokens and triggers the LLM inference to generate LLM output (the completion tokens). The queue manager writes both the incoming message and the generated LLM output to recall storage (the MemGPT message database). When messages in recall storage are retrieved via a MemGPT function call, the queue manager appends them to the back of

图 *1*。 MemGPT（左）在收到关于有限上下文空间的系统警报后，将数据写入持久存储。

图*2*。 MemGPT（左）能够搜索超出上下文的数据，将相关信息引入当前上下文窗口。

通过虚拟内存，操作系统通过页面换入换出将溢出数据存储到磁盘，并在应用程序访问时（通过页面错误）将数据重新加载到内存中，从而提供一种比实际物理（即主）内存更丰富的内存资源的错觉。为了提供类似的更长上下文长度的错觉（类似于虚拟内存），我们允许LLM通过“LLM操作系统”管理其自身上下文中的内容（类似于物理内存），我们称之为MemGPT。MemGPT使LLM能够检索缺失的相关历史数据（在上下文中未包含的部分），并且还能将不那么相关的数据从上下文中驱逐出去，存入外部存储系统。图3展示了MemGPT的组成部分。

结合使用内存层级、操作系统功能和事件驱动控制流，MemGPT能够利用具有有限上下文窗口的LLMs处理无限制的上下文。为了展示我们新型OS启发式LLM系统的实用性，我们在两个受限于有限上下文的领域对MemGPT进行了评估：文档分析，在该领域中，标准文本文件的长度很快就会超过现代LLMs的输入容量；以及对话代理，在有限对话窗口限制下的LLMs缺乏上下文感知、角色一致性和长时记忆，尤其在长时间对话中表现明显。在这两种场景中，MemGPT都能克服有限上下文的限制，优于现有的基于LLMs的方法。

2. MemGPT（MemoryGPT）

MemGPT的类操作系统多层次记忆架构区分了两种主要的记忆类型：**主上下文**（类比于主存/物理内存/RAM）和**外部上下文**（类比于磁盘存储/磁盘存储器）。主上下文由LLM 提示标记组成——在主上下文中的任何内容都被视为上下文内，可以在推理过程中被LLM处理器访问。外部上下文指的是任何存储在LLM固定上下文窗口之外的信息。这些超出上下文的数据

必须始终将其明确地移动到主上下文中，以便在推理过程中将其传递给LLM处理器。MemGPT提供了函数调用，允许LLM处理器在无需用户干预的情况下管理其自身的内存。

2.1. 主要内容 (*prompt tokens*)

MemGPT中的提示令牌被划分为三个连续的部分：**系统指令**、**工作上下文** 和 **FIFO队列**。系统指令为只读（静态），包含关于MemGPT控制流程、不同内存层级的预期用途以及如何使用MemGPT功能（例如如何检索超出上下文的数据）的信息。工作上下文是一个固定大小的读写块，无结构文本，仅通过MemGPT函数调用进行写入。在对话环境中，工作上下文旨在用来存储关键事实、偏好以及关于用户和代理所采用角色的其他重要信息，从而使代理能够与用户流畅对话。FIFO队列存储消息的滚动历史，包括代理与用户之间的消息、系统消息（例如内存警告）以及函数调用的输入和输出。FIFO队列中的第一个索引存储一个系统消息，包含对已从队列中剔除的消息的递归摘要。

2.2. 队列管理器

队列管理器管理**回忆存储**和**先进先出队列**中的消息。当系统接收到新消息时，队列管理器将传入的消息添加到先进先出队列中，连接提示符标记并触发LLM推理以生成LLM输出（完成标记）。队列管理器将传入的消息和生成的LLM输出都写入**回忆存储**（MemGPT消息数据库）。当通过MemGPT函数调用检索回忆存储中的消息时，队列管理器会将它们追加到队列的后端。

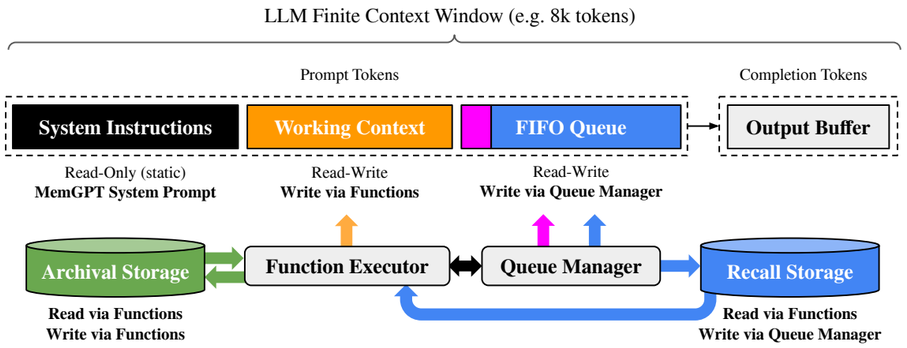

Figure 3. In MemGPT, a fixed-context LLM processor is augmented with a hierarchical memory system and functions that let it manage its own memory. The LLM's prompt tokens (inputs), or *main context*, consist of the system instructions, working context, and a FIFO queue. The LLM completion tokens (outputs) are interpreted as function calls by the function executor. MemGPT uses functions to move data between main context and *external context* (the archival and recall storage databases). The LLM can request immediate follow-up LLM inference to chain function calls together by generating a special keyword argument (`request_heartbeat=true`) in its output; function chaining is what allows MemGPT to perform multi-step retrieval to answer user queries.

the queue to reinsert them into the LLM's context window.

The queue manager is also responsible for controlling context overflow via a queue eviction policy. When the prompt tokens exceed the 'warning token count' of the underlying LLM's context window (e.g. 70% of the context window), the queue manager inserts a system message into the queue warning the LLM of an impending queue eviction (a 'memory pressure' warning) to allow the LLM to use MemGPT functions to store important information contained in the FIFO queue to working context or **archival storage** (a read/write database storing arbitrary length text objects). When the prompt tokens exceed the 'flush token count' (e.g. 100% of the context window), the queue manager flushes the queue to free up space in the context window: the queue manager evicts a specific count of messages (e.g. 50% of the context window), generates a new recursive summary using the existing recursive summary and evicted messages. Once the queue is flushed, the evicted messages are no longer in-context and immediately viewable to the LLM, however they are stored indefinitely in recall storage and readable via MemGPT function calls.

2.3. Function executor (handling of *completion tokens*)

MemGPT orchestrates data movement between main context and external context via function calls that are generated by the LLM processor. Memory edits and retrieval are entirely self-directed: MemGPT autonomously updates and searches through its own memory based on the current context. For instance, it can decide when to move items between contexts (e.g. when the conversation history is becoming too long, as show in Figure 1) and modify its main context to better reflect its evolving understanding of its current objectives and responsibilities (as shown in Figure 3). We implement self-directed editing and retrieval by providing explicit instructions within the system instructions that guide the LLM on how to interact with the MemGPT memory systems. These instructions comprise two main components: (1) a detailed description of the memory hierarchy and their respective utilities, and (2) a function schema (complete with their natural language descriptions) that the system can call to access or modify its memory.

During each inference cycle, LLM processor takes main context (concatenated into a single string) as input, and generates an output string. This output string is parsed by MemGPT to ensure correctness, and if the parser validates the function arguments the function is executed. The results, including any runtime errors that occur (e.g. trying to add to main context when it is already at maximum capacity), are then fed back to the processor by MemGPT. This feedback loop enables the system to learn from its actions and adjust its behavior accordingly. Awareness of context limits is a key aspect in making the self-editing mechanism work effectively, to this end MemGPT prompts the processor with warnings regarding token limitations to guide its memory management decisions. Additionally, our memory retrieval mechanisms are designed to be cognizant of these token constraints and implement pagination to prevent retrieval calls from overflowing the context window.

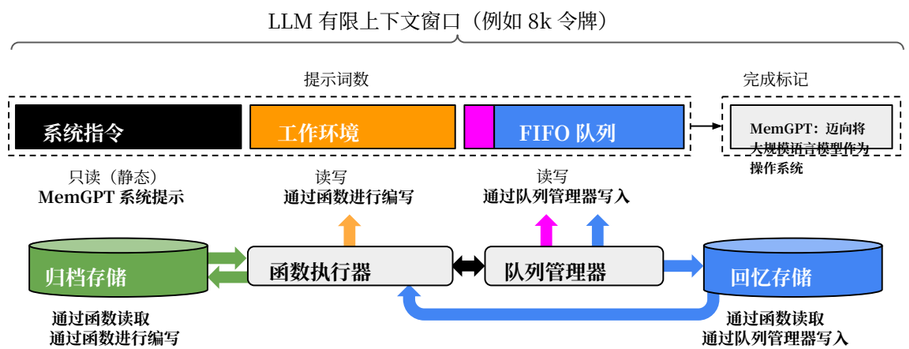

图*3*。在MemGPT中，固定上下文的大规模语言模型处理器（LLM processor）通过引入层级式存储系统和管理自身存储的功能得到增强。LLM的提示标记（输入），即主上下文，由系统指令、工作上下文和先进先出（FIFO）队列组成。LLM的完成标记（输出）被解释为函数调用，由函数执行器执行。MemGPT利用函数在主上下文与外部上下文（归档与检索存储数据库）之间传输数据。LLM可以通过在输出中生成特殊关键词参数（request heartbeat=true）请求立即进行后续的LLM推理，以实现函数调用的链式连接。函数链式调用使得MemGPT能够进行多步检索，从而回答用户查询。

队列将它们重新插入到大规模语言模型的上下文窗口中。

队列管理器还负责通过队列驱逐策略控制上下文溢出。当提示词超过底层LLM的上下文窗口的“警告词数”阈值（例如，占上下文窗口的70%）时，队列管理器会在队列中插入一条系统消息，警告LLM即将发生队列驱逐（“内存压力”警告），以便LLM利用MemGPT功能将FIFO队列中包含的重要信息存储到工作上下文或**存档存储**（一种存储任意长度文本对象的读写数据库）。当提示词超过“刷新词数”阈值（例如，占上下文窗口的100%）时，队列管理器会刷新队列以释放上下文窗口中的空间：驱逐特定数量的消息（例如，占上下文窗口的50%），并使用现有的递归摘要和被驱逐的消息生成新的递归摘要。一旦队列被刷新，被驱逐的消息将不再在上下文中，LLM无法立即访问，但它们会被无限期存储在回忆存储中，并可以通过MemGPT功能调用读取。

2.3. *completion tokens*的函数执行器

MemGPT 通过由大规模语言模型（LLM）处理器生成的函数调用，在主上下文与外部上下文之间协调数据迁移。内存的编辑与检索完全由系统自主完成：MemGPT 根据当前上下文自主更新和搜索其自身的内存。例如，它可以自主决定何时在不同上下文之间移动条目（例如，当对话历史...

tory 变得过长，如图1所示，并修改其主要内容以更好地反映其对当前目标和责任的不断演变的理解（如图3所示）。我们通过在系统指令中提供明确的指导，实施自主编辑和检索，指导 LLM 如何与 MemGPT 内存系统交互。这些指令包括两个主要组成部分：(1) 内存层级及其各自功能的详细描述，以及 (2) 一个函数架构（附有自然语言描述），系统可以调用该架构以访问或修改其内存。

在每个推理周期中，LLM处理器将主上下文（连接成一个字符串）作为输入，并生成一个输出字符串。该输出字符串由MemGPT进行解析以确保正确性，如果解析器验证了函数参数，则执行该函数。随后，MemGPT将包括任何运行时错误（例如在主上下文已达最大容量时尝试添加内容）的结果反馈给处理器。这个反馈循环使系统能够从其操作中学习并相应调整行为。对上下文限制的意识是使自我编辑机制有效运作的关键，为此，MemGPT会向处理器发出关于令牌限制的警告，以指导其内存管理决策。此外，我们的内存检索机制设计时考虑到这些令牌限制，并实现分页，以防止检索调用溢出上下文窗口。

Table 1. Comparing context lengths of commonly used models and LLM APIs (data collected 1/2024). *Approximate message count assuming a preprompt of 1k tokens, and an average message size of ∼50 tokens (∼250 characters). 'Open' means the model is open-source or open-weights (vs only available behind an API).

Model / API name	Open?	Context Window Tokens	*Messages
Llama (1)	✓	2k	20
Llama 2	✓	4k	60
GPT-3.5 Turbo (release)	✗	4k	60
Mistral 7B	✓	8k	140
GPT-4 (release)	✗	8k	140
GPT-3.5 Turbo	✗	16k	300
GPT-4	✗	32k	∼600
Claude 2	✗	100k	∼2000
GPT-4 Turbo	✗	128k	∼2600
Yi-34B-200k	✓	200k	∼4000

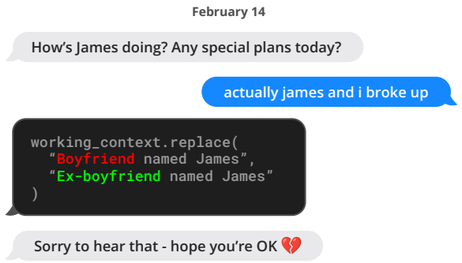

Figure 4. An example conversation snippet where MemGPT (left) updates stored information. Here the information is stored in working context memory (located within the prompt tokens).

2.4. Control flow and function chaining

In MemGPT, *events* trigger LLM inference: events are generalized inputs to MemGPT and can consist of user messages (in chat applications), system messages (e.g. main context capacity warnings), user interactions (e.g. an alert that a user just logged in, or an alert that they finished uploading a document), and timed events that are run on a regular schedule (allowing MemGPT to run 'unprompted' without user intervention). MemGPT processes events with a parser to convert them into plain text messages that can be appended to main context and eventually be fed as input into the LLM processor.

Many practical tasks require calling multiple functions in sequence, for example, navigating through multiple pages of results from a single query or collating data from different documents in main context from separate queries. Function chaining allows MemGPT to execute multiple function calls sequentially before returning control to the user. In MemGPT, functions can be called with a special flag that requests control be immediately returned to the processor after the requested function completes execution. If this flag is present, MemGPT will add the function output to main context and (as opposed to pausing processor execution). If this flag is not present (a *yield*), MemGPT will not run the LLM processor until the next external event trigger (e.g. a user message or scheduled interrupt).

3. Experiments

We assess MemGPT in two long-context domains: conversational agents and document analysis. For conversational agents, we expand the existing Multi-Session Chat dataset (Xu et al., 2021) and introduce two new dialogue tasks that evaluate an agent's ability to retain knowledge across long conversations. For document analysis, we benchmark MemGPT on existing tasks from (Liu et al., 2023a) for question answering and key-value retrieval over lengthy documents. We also propose a new nested key-value retrieval task requiring collating information across multiple data sources, which tests the ability of an agent to collate information from multiple data sources (multi-hop retrieval). We publicly release our augmented MSC dataset, nested KV retrieval dataset, and a dataset of embeddings for 20M Wikipedia articles to facilitate future research. Our code for the benchmarks is available at https://research.memgpt.ai.

Implementation details. When discussing OpenAI models, unless otherwise specified 'GPT-4 Turbo' refers to the specific `gpt-4-1106-preview` model endpoint (context window of 128,000), 'GPT-4' refers to `gpt-4-0613` (context window of 8,192), and 'GPT-3.5 Turbo' refers to `gpt-3.5-turbo-1106` (context window of 16,385). In experiments, we run MemGPT with all baseline models (GPT-4, GPT-4 Turbo, and GPT 3.5) to show how the underlying model performance affects MemGPT's.

3.1. MemGPT for conversational agents

Conversational agents like virtual companions and personalized assistants aim to engage users in natural, long-term interactions, potentially spanning weeks, months, or even years. This creates challenges for models with fixed-length contexts, which can only reference a limited history of the conversation. An 'infinite context' agent should seamlessly handle continuous exchanges without boundary or reset. When conversing with a user, such an agent must satisfy two key criteria: (1) Consistency - The agent should maintain conversational coherence. New facts, preferences, and events mentioned should align with prior statements from both the user and agent. (2) Engagement - The agent should draw on long-term knowledge about the user to personalize

表 *1.* 常用模型与LLM API的上下文长度比较（数据采集于2024年1月）。*假设预提示为1k个标记，平均消息大小为∼50 个标记（∼250 个字符），大致消息数。'Open'表示模型为开源或开权重（与仅通过API提供的模型相对）。

模型 /API名称	开启?	上下文窗口 Token	消息
Llama (1) ✓	2k		20
Llama 2	✓	4k	60
GPT-3.5 Turbo（发布）	✗	4k	60
Mistral 7B	✓	8k	140
GPT-4（发布）	✗	8k	140
GPT-3.5 Turbo	✗	16k	300
MemGPT：迈向将大规模语言模型作为操作系统	✗	32k	∼600
Claude 2	✗	100k	∼2000
GPT-4 Turbo	✗	128k	∼2600
Yi-34B-200k	✓	200k	∼4000

图*4*。一个示例对话片段，MemGPT（左）更新存储的信息。这里的信息存储在工作上下文记忆中（位于提示令牌内）。

2.4. 控制流与函数链式调用

在 MemGPT 中，事件触发大规模语言模型（LLM）推理：事件是对 MemGPT 的泛化输入，可以包括用户消息（在聊天应用中）、系统消息（例如主上下文容量警告）、用户交互（例如用户刚登录的提醒，或上传文档完成的通知），以及按计划定期运行的定时事件（允许 MemGPT 在没有用户干预的情况下"自主"运行）。MemGPT 使用解析器处理事件，将其转换为纯文本消息，这些消息可以附加到主上下文中，最终作为输入提供给大规模语言模型处理器。

许多实际任务需要依次调用多个函数，例如，从单个查询中导航多个结果页面，或从不同的查询中整理主上下文中的数据。函数链允许MemGPT在将控制权返回给用户之前，连续执行多个函数调用。在MemGPT中，可以使用带有特殊标志的函数调用，该标志请求在所请求的函数完成执行后立即将控制权返回给处理器。如果存在此标志，MemGPT将把函数输出添加到主上下文中（而不是暂停处理器的执行）。如果没有此标志（即*yield*），MemGPT将不会运行LLM处理器，直到下一次外部事件触发（例如用户消息或计划中断）。

3. 实验

我们在两个长上下文领域评估MemGPT：对话代理和文档分析。对于对话代理，我们扩展了现有的多会话聊天数据集（Xu et al., 2021），并引入了两个新的对话任务，以评估代理的知识保持能力。

在长对话中。对于文档分析，我们在（Liu et al., 2023a）提出的现有任务上对MemGPT进行了基准测试，包括长文档的问答和关键值检索。我们还提出了一项新的嵌套关键值检索任务，要求整合多个数据源的信息，测试代理在多跳检索中的信息整合能力。我们公开发布了扩展的MSC数据集、嵌套KV检索数据集以及用于20M维基百科文章的嵌入数据集，以促进未来的研究。我们的基准测试代码可在https://research.memgpt.ai获取。

实现细节。在讨论OpenAI模型时，除非另有说明，"GPT-4 Turbo"指的是特定的gpt-4-1106-preview模型端点（上下文窗口为128,000），"GPT-4"指的是gpt-4-0613（上下文窗口为8,192），而"GPT-3.5 Turbo"指的是gpt-3.5-turbo-1106（上下文窗口为16,385）。在实验中，我们使用所有基线模型（GPT-4、GPT-4 Turbo和GPT-3.5）运行MemGPT，以展示基础模型性能如何影响MemGPT的表现。

3.1. 用于对话代理的MemGPT

对话代理如虚拟伴侣和个性化助手旨在与用户进行自然、长期的互动，可能持续数周、数月甚至数年。这为具有固定长度上下文的模型带来了挑战，因为它们只能引用有限的对话历史。一个"无限上下文"代理应能够无缝处理连续的交流，无边界或重置。当与用户对话时，这样的代理必须满足两个关键标准：(1) 一致性——代理应保持对话的连贯性。提及的新事实、偏好和事件应与用户和代理之前的陈述保持一致。(2) 参与度——代理应利用关于用户的长期知识进行个性化。

Table 2. **Deep memory retrieval (DMR) performance.** In this task, the agent is asked a specific question about a topic discussed in a prior conversation (sessions 1–5). The agent's response is scored against the gold answer. MemGPT significantly outperforms the fixed-context baselines.

Model	Accuracy ⇑	ROUGE-L (R) ⇑
GPT-3.5 Turbo	38.7%	0.394
+ MemGPT	66.9%	0.629
GPT-4	32.1%	0.296
+ MemGPT	92.5%	0.814
GPT-4 Turbo	35.3%	0.359
+ **MemGPT**	**93.4%**	**0.827**

Table 3. **Conversation opener performance.** The agent's conversation opener is evaluated using similarity scores to the gold persona labels (SIM-1/3) and to the human-created opener (SIM-H). MemGPT is able to exceed the performance of the human-created conversation opener with a variety of underlying models.

Method	⇑ SIM-1	SIM-3	SIM-H
Human	0.800	0.800	1.000
GPT-3.5 Turbo	0.830	0.812	**0.817**
GPT-4	**0.868**	**0.843**	0.773
GPT-4 Turbo	0.857	0.828	0.767

responses. Referencing prior conversations makes dialogue more natural and engaging.

We therefore assess our proposed system, MemGPT, on these two criteria: (1) Does MemGPT leverage its memory to improve conversation consistency? Can it remember relevant facts, preferences, and events from past interactions to maintain coherence? (2) Does MemGPT produce more engaging dialogue by taking advantage of memory? Does it spontaneously incorporate long-range user information to personalize messages? By evaluating on consistency and engagement, we can determine how well MemGPT handles the challenges of long-term conversational interaction compared to fixed-context baselines. Its ability to satisfy these criteria will demonstrate whether unbounded context provides meaningful benefits for conversational agents.

Dataset. We evaluate MemGPT and our fixed-context baselines on the Multi-Session Chat (MSC) dataset introduced by Xu et al. (2021), which contains multi-session chat logs generated by human labelers, each of whom was asked to play a consistent persona for the duration of all sessions. Each multi-session chat in MSC has five total sessions, and each session consists of a roughly a dozen messages. As part of our consistency experiments, we created a new session (session 6) that contains a single question-answer response pair between the same two personas.

3.1.1. Deep memory retrieval task (consistency).

We introduce a new 'deep memory retrieval' (DMR) task based on the MSC dataset designed to test the consistency of a conversational agent. In DMR, the conversational agent is asked a question by the user that explicitly refers back to a prior conversation and has a very narrow expected answer range. We generated the DMR question-answer (QA) pairs using a separate LLM that was instructed to write a question from one user to another that could only be answered correctly using knowledge gained from the past sessions (see Appendix for further details).

We evaluate the quality of the generated response against the 'gold response' using ROUGE-L scores (Lin, 2004) and an 'LLM judge', which is instructed to evaluate whether or not the generated response is consistent with the gold response (GPT-4 has been shown to have high agreement with human evaluators (Zheng et al., 2023)). In practice, we notice that the generated responses (from both MemGPT and the baselines) were generally more verbose than the gold responses. We use the ROUGE-L recall (R) metric to account for the verbosity of the generated agent replies compared to the relatively short gold answer labels.

MemGPT utilizes memory to maintain coherence: Table 2 shows the performance of MemGPT vs the fixed-memory baselines. We compare MemGPT using different underlying LLMs, and compare against using the base LLM without MemGPT as a baseline. The baselines are able to see a lossy summarization of the past five conversations to mimic an extended recursive summarization procedure, while MemGPT instead has access to the full conversation history but must access it via paginated search queries to recall memory (in order to bring them into main context). In this task, we see that MemGPT clearly improves the performance of the underlying base LLM: there is a clear drop in both accuracy and ROUGE scores when going from MemGPT to the corresponding LLM baselines.

3.1.2. Conversation opener task (engagement).

In the 'conversation opener' task we evaluate an agent's ability to craft engaging messages to the user that draw from knowledge accumulated in prior conversations. To evaluate the 'engagingness' of a conversation opener using the MSC dataset, we compare the generated opener to the gold personas: an engaging conversation opener should draw from one (or several) of the data points contained in the persona, which in MSC effectively summarize the knowledge accumulated throughout all prior sessions. We also compare to the human-generated gold opener, i.e., the

表2。 **深度记忆检索（DMR）性能。** 在此任务中，代理被问及关于之前会话（第1–5次会话）中讨论的主题的具体问题。代理的回答将与标准答案进行评分。MemGPT在此任务中显著优于固定上下文基线。

模型	准确性 ⇑	ROUGE-L (R) ⇑
MemGPT：迈向将大型语言模型作为操作系统	38.7%	0.394
MemGPT	66.9%	0.629
MemGPT：迈向将大型语言模型作为操作系统	32.1%	0.296
MemGPT	92.5%	0.814
GPT-4 Turbo	35.3%	0.359
MemGPT	**93.4%**	**0.827**

表3。 **对话开启者性能。** 代理的对话开启者通过与黄金人物标签的相似度得分（SIM-1/3）以及与人工创建的开启者的相似度（SIM-H）进行评估。在多种基础模型下，MemGPT能够超越人工创建的对话开启者的性能。

方法	SIM-1	SIM-3	SIM-H
请提供需要翻译的文本内容。	0.800	0.800	1.000
MemGPT：迈向将大型语言模型作为操作系统	0.830	0.812	**0.817**
MemGPT：迈向将大规模语言模型作为操作系统	**0.868**	**0.843**	0.773
GPT-4 Turbo	0.857	0.828	0.767

回应。参考先前的对话可以使对话更加自然和生动。

因此，我们在这两个标准上评估我们提出的系统MemGPT：(1) MemGPT是否利用其记忆来提高对话的一致性？它是否能够记住过去交互中的相关事实、偏好和事件以保持连贯性？(2) MemGPT是否通过利用记忆产生更具吸引力的对话？它是否能自发地融入长距离的用户信息以实现个性化信息传递？通过在一致性和吸引力方面的评估，我们可以判断MemGPT在应对长期对话交互挑战方面的表现如何，相较于固定上下文的基线。其满足这些标准的能力将展示无界上下文是否为对话代理带来有意义的益处。

数据集。我们在由Xu等人（2021）引入的多会话聊天（MSC）数据集上评估了MemGPT及我们的固定上下文基线，该数据集包含由人工标注者生成的多会话聊天记录，每位标注者被要求在所有会话中扮演一致的角色。MSC中的每个多会话聊天共有五个会话，每个会话由大约十余条消息组成。作为我们一致性实验的一部分，我们创建了一个新的会话（会话6），其中包含同一对角色之间的单个问答对。

3.1.1. 深度记忆检索任务（一致性）

我们引入了一项基于MSC数据集的全新“深度记忆检索”(DMR)任务，旨在测试对话代理的一致性。在DMR中，用户向对话代理提出一个明确回溯至先前对话的问题，并且该问题的预期答案范围非常狭窄。我们使用另一台被指示撰写用户之间问题的独立大型语言模型（LLM）生成了DMR的问答（QA）对，这些问题只能通过从过去获得的知识才能正确回答

会话（详见附录）

我们使用ROUGE-L分数（Lin, 2004）和“LLM评判”对生成回复的质量进行评估，后者被指示评估生成回复是否与“金标准回复”一致（已证明GPT-4在与人类评估者的一致性方面具有较高的契合度（Zheng et al., 2023））。在实践中，我们注意到生成的回复（无论是MemGPT还是基线模型）通常比金标准回复更为冗长。我们采用ROUGE-L召回（R）指标，以考虑生成代理回复相较于相对简短的金标准答案标签的冗长程度。

MemGPT 利用记忆保持连贯性：表2显示了MemGPT与固定记忆基线的性能对比。我们比较了使用不同底层LLMs的MemGPT，并以不使用MemGPT的基础LLM作为对比基线。基线模型能够看到过去五次对话的有损摘要，以模拟扩展的递归摘要过程，而MemGPT则可以访问完整的对话历史，但必须通过分页搜索查询来调取记忆（以将其引入主上下文中）。在此任务中，我们观察到MemGPT明显提升了底层基础LLM的性能：从MemGPT切换到相应的LLM基线时，准确率和ROUGE分数都出现了明显下降。

3.1.2. 对话开启任务（参与度）

在“对话开启者”任务中，我们评估代理人根据先前对话中积累的知识，设计引人入胜的消息与用户互动的能力。为了评估使用MSC数据集的“引人入胜性”，我们将生成的开启者与黄金人设进行比较：一个引人入胜的对话开启者应当借鉴人设中包含的一个（或多个）数据点，而在MSC中，这些数据点有效总结了所有先前会话中积累的知识。我们还将与人工生成的黄金开启者进行比较，即

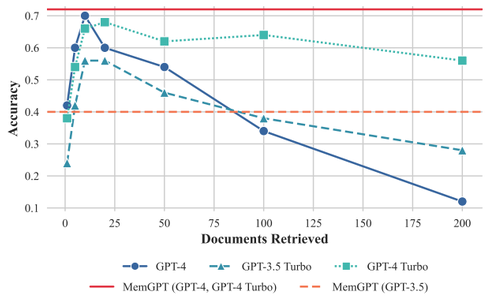

Figure 5. **Document QA task performance.** MemGPT's performance is unaffected by increased context length. Methods such as truncation can extend the effective context lengths of fixed length models such as GPT-4, but such compression methods will lead to performance degradation as the necessary compression grows. Running MemGPT with GPT-4 and GPT-4 Turbo have equivalent results on this task.

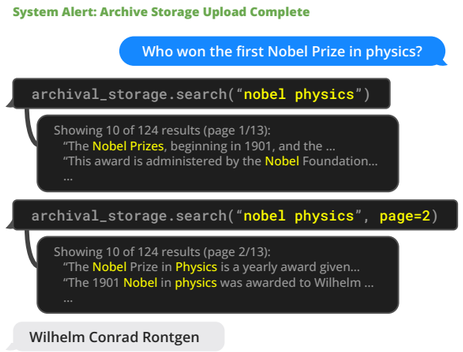

Figure 6. An example of MemGPT (left) solving the document QA task. A database of Wikipedia documents is uploaded to archival storage. MemGPT queries archival storage via function calling, which pulls paginated search results into main context.

first response in the following session. We report the CSIM scores of MemGPT's openers in Table 3. We test several variations of MemGPT using different base LLMs.

MemGPT utilizes memory to increase engagement: As seen in Table 3, MemGPT is able to craft engaging openers that perform similarly to and occasionally exceed the hand-written human openers. We observe that MemGPT tends to craft openers that are both more verbose and cover more aspects of the persona information than the human baseline. Additionally, we can see the storing information in working context is key to generating engaging openers.

3.2. MemGPT for document analysis

Document analysis also faces challenges due to the limited context windows of today's transformer models. As shown in Table 1, both open and closed source models suffer from constrained context length (up to 128k tokens for OpenAI's models). However many documents easily surpass these lengths; for example, legal or financial documents such as Annual Reports (SEC Form 10-K) can easily pass the million token mark. Moreover, many real document analysis tasks require drawing connections across multiple such lengthy documents. Anticipating these scenarios, it becomes difficult to envision blindly scaling up context as a solution to the fixed-context problem. Recent research (Liu et al., 2023a) also raises doubts about the utility of simply scaling contexts, since they find uneven attention distributions in large context models (the model is more capable of recalling information at the beginning or end of its context window, vs tokens in the middle). To enable reasoning across documents, more flexible memory architectures like MemGPT are needed.

3.2.1. Multi-document question-answering.

To evaluate MemGPT's ability to analyze documents, we benchmark MemGPT against fixed-context baselines on the retriever-reader document QA task from Liu et al. (2023a). In this task, a question is selected from the NaturalQuestions-Open dataset, and a retriever selects relevant Wikipedia documents for the question. A reader model (the LLM) is then fed these documents as input, and is asked to use the provided documents to answer the question. Similar to Liu et al. (2023a), we evaluate reader accuracy as the number of retrieved documents K increases.

In our evaluation setup, both the fixed-context baselines and MemGPT use the same retriever, which selects the top K documents according using similarity search (cosine distance) on OpenAI's `text-embedding-ada-002` embeddings. We use MemGPT's default storage settings which uses PostgreSQL for archival memory storage with vector search enabled via the pgvector extention. We precompute embeddings and load them into the database, which uses an HNSW index to enable approximate, sub-second query times. In MemGPT, the entire embedding document set is loaded into archival storage, and the retriever naturally emerges via the archival storage search functionality (which performs vector search based on cosine similarity). In the fixed-context baselines, the top-K documents are fetched using the retriever independently from the LLM inference, similar to the original retriever-reader setup in Liu et al. (2023a).

We use a dump of Wikipedia from late 2018, following past work on NaturalQuestions-Open (Izacard & Grave, 2020;

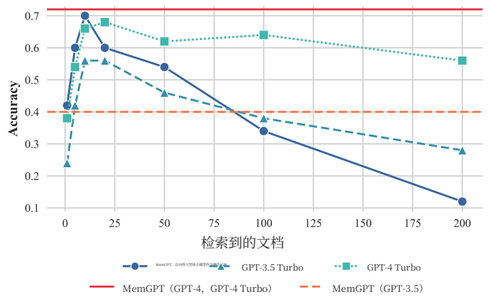

图 *5*。**文档问答任务表现**。MemGPT的性能不受增加的上下文长度影响。诸如截断等方法可以扩展固定长度模型（如GPT-4）的有效上下文长度，但随着所需压缩的增加，这些压缩方法将导致性能下降。在此任务中，使用GPT-4和GPT-4 Turbo运行MemGPT的结果是等效的。

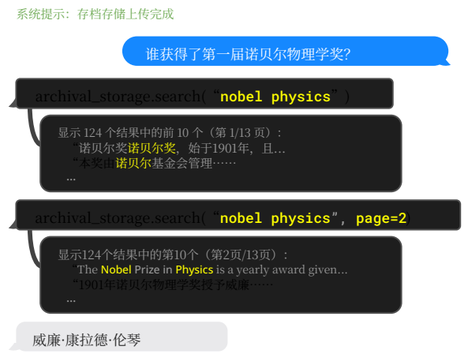

图*6*。MemGPT（左）解决文档问答任务的示例。一个维基百科文档数据库被上传到存档存储中。MemGPT通过函数调用查询存档存储，将分页搜索结果拉入主上下文。

我们在表3中报告了MemGPT的开启器的CSIM分数。我们使用不同的基础大型语言模型（LLMs）测试了MemGPT的几种变体。

MemGPT 利用记忆增强互动性：如表3所示，MemGPT能够设计出与人工手写开场白表现相似甚至有时优于人工开场白的引人入胜的开场白。我们观察到，MemGPT倾向于设计出比人工基线更为冗长且涵盖更多人物信息方面的开场白。此外，我们可以看到，将信息存储在工作上下文中是生成引人入胜的开场白的关键。

3.2. 用于文档分析的MemGPT

文档分析也面临着由于当今变换器模型的有限上下文窗口而带来的挑战。如表1所示，无论是开源还是闭源模型，都受到上下文长度限制（OpenAI的模型最多支持128k标记）。然而，许多文档的长度远远超过这些限制；例如，法律或财务文件，如年度报告（SEC表格10-K），很容易超过一百万标记。此外，许多实际的文档分析任务需要在多个如此长的文档之间建立联系。预见到这些场景，盲目扩大上下文规模以解决固定上下文问题变得困难。近期研究（Liu等，2023a）也对单纯扩大上下文规模的实用性提出了质疑，因为他们发现大型上下文模型中的注意力分布不均（模型更擅长回忆其上下文窗口的开头或结尾的信息，而对中间的标记则较弱）。为了实现跨文档推理，更加灵活的记忆架构变得尤为重要。

需要 MemGPT。

3.2.1. 多文档问答

为了评估MemGPT分析文档的能力，我们在Liu等人（2023a）提出的检索-阅读文档问答任务中，将MemGPT与固定上下文基线进行了对比。在该任务中，从NaturalQuestions-Open数据集中选择一个问题，检索器为该问题选择相关的维基百科文档。然后，将这些文档作为输入提供给阅读模型（即LLM），并要求其利用提供的文档回答问题。与Liu等人（2023a）类似，我们评估了随着检索到的文档数量 K 增加，阅读器的准确率。

在我们的评估设置中，固定上下文基线和MemGPT使用相同的检索器，后者通过在OpenAI的text-embedding-ada-002嵌入向量上进行相似度搜索（余弦距离）选择前K个文档。我们采用MemGPT的默认存储设置，使用PostgreSQL进行归档内存存储，并通过pgvector扩展启用向量搜索。我们预先计算嵌入向量并将其加载到数据库中，数据库采用HNSW索引以实现近似的亚秒级查询时间。在MemGPT中，整个嵌入文档集被加载到归档存储中，检索器自然通过归档存储的搜索功能（基于余弦相似度进行向量搜索）实现。在固定上下文基线中，前K个文档由检索器独立于LLM推理进行获取，类似于Liu等人（2023a）中的原始检索-阅读器设置。

我们使用了2018年末的维基百科数据快照，沿袭了之前关于NaturalQuestions-Open的研究（Izacard & Grave, 2020；）

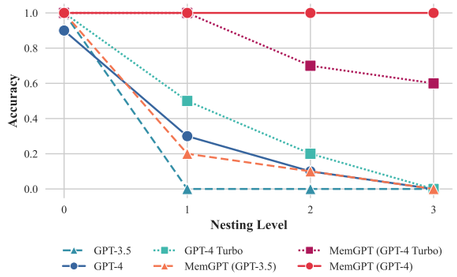

Figure 7. **Nested KV retrieval task performance.** MemGPT is the only approach that is able to consistently complete the nested KV task beyond 2 nesting levels. While GPT-4 Turbo performs better as a baseline, MemGPT with GPT-4 Turbo performs worse than MemGPT with GPT-4.

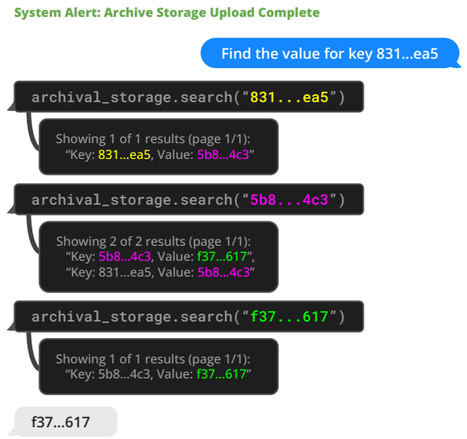

Figure 8. An example of MemGPT (left) solving the nested KV task (UUIDs shortened for readability). In this particular example, the key-value pair has two nesting levels: `831..ea5` → `5b8..4c3` → `f37...617`. The MemGPT agent returns the final answer when a query for the final value (`f37...617`) only returns one result, indicating that it is not also a key.

Izacard et al., 2021), and sampled a subset of 50 questions for evaluation. Both the sampled questions and embedded Wikipedia passages are publicaly released. We evaluate the performance of both MemGPT and baselines with an LLM-judge, to ensure that the the answer is properly derived from the retrieved documents and to avoid non-exact string matches being considered incorrect.

We show the results for the document QA task in Figure 5. The fixed-context baselines performance is capped roughly at the performance of the retriever, as they use the information that is presented in their context window (e.g. if the embedding search retriever fails to surface the gold article using the provided question, the fixed-context baselines are guaranteed to never see the gold article). By contrast, MemGPT is effectively able to make multiple calls to the retriever by querying archival storage, allowing it to scale to larger effective context lengths. MemGPT actively retrieves documents from its archival storage (and can iteratively page through results), so the total number of documents available to MemGPT is no longer limited by the number of documents that fit within the LLM processor's context window.

The document QA task is challenging for all methods due to the limitations of embedding-based similarity search. We observe that the golden document for chosen question (as annotated by NaturalQuestions-Open) often appears outside of the first dozen retrieved results, if not even further. The retriever performance translates directly to the fixed-context baseline results: GPT-4's accuracy is relatively low with few retrieved documents, and continues to improve as additional documents are added to the context window, as it correctly limits itself to answering questions based on information in retrieved documents. While MemGPT is theoretically not limited by sub-optimal retriever performance (even if the embedding-based ranking is noisy, as long as the full retriever ranking contains the gold document it can still be found with enough retriever calls via pagination), we observe that MemGPT will often stop paging through retriever results before exhausting the retriever database.

To evaluate the fixed-context baselines against MemGPT past their default context lengths, we truncate the document segments returned by the retriever to fix the same number of documents into the available context. As expected, document truncation reduces accuracy as documents shrink as the chance of the relevant snippet (in the gold document) being omitted grows, as shown in Figure 5. MemGPT has significantly degraded performance using GPT-3.5, due to its limited function calling capabilities, and performs best using GPT-4.

3.2.2. Nested key-value retrieval (KV).

We introduce a new task based on the synthetic Key-Value retrieval proposed in prior work (Liu et al., 2023a). The goal of this task is to demonstrate how MemGPT can collate information from multiple data sources. In the original KV task, the authors generated a synthetic dataset of key-value pairs, where each key and value is a 128-bit UUID (universally unique identifier). The agent is then given a key, and asked to return the associated value for the key. We create a version of the KV task, *nested KV retrieval*,

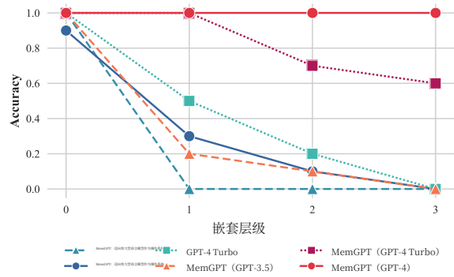

图7。 **嵌套KV检索任务性能**。 MemGPT是唯一能够在超过2层嵌套的情况下持续完成嵌套KV任务的方法。虽然GPT-4 Turbo作为基线表现更佳，但使用GPT-4 Turbo的MemGPT的表现不如使用GPT-4的MemGPT。

Izacard 等人，2021 年，抽取了 50 个问题的子集进行评估。所抽取的问题和嵌入的维基百科段落均已公开发布。我们使用 LLM 评判对 MemGPT 和基线模型的性能进行评估，以确保答案确实源自检索到的文档，并避免将非精确字符串匹配误判为错误。

我们在图5中展示了文档问答任务的结果。固定上下文基线的性能大致被检索器的性能所限制，因为它们仅利用在其上下文窗口中呈现的信息（例如，如果嵌入搜索检索器未能通过提供的问题找到黄金文章，固定上下文基线就无法看到黄金文章）。相比之下，MemGPT实际上能够通过查询存档存储多次调用检索器，从而扩展到更大的有效上下文长度。MemGPT主动从其存档存储中检索文档（并可以迭代浏览结果），因此，MemGPT可用的文档总数不再受限于能够容纳在LLM处理器上下文窗口中的文档数量。

MemGPT：迈向将大规模语言模型作为操作系统

图8。 MemGPT（左）解决嵌套KV任务的示例（为便于阅读，UUID已缩短）。在此特定示例中，键值对具有两个嵌套层级：`831` → `5..ea5` → `5b..4c3` → `f...617`。当对最终值（f37...617）进行查询时，MemGPT代理返回最终答案，且只返回一个结果，表明该结果既不是键也不是值。

检索器性能（即使基于嵌入的排序存在噪声，只要完整的检索器排序中包含了黄金文档，通过分页调用检索器仍然可以找到它），我们观察到，MemGPT 通常会在用尽检索器数据库之前就停止分页浏览检索结果。

为了评估固定上下文基线在超出其默认上下文长度的情况下与MemGPT的表现，我们将检索器返回的文档片段进行截断，以确保在可用上下文中包含相同数量的文档。如预期所示，文档截断会降低准确率，因为随着文档的缩减，相关片段（在金标准文档中）被遗漏的可能性增加，如图5所示。使用GPT-3.5时，MemGPT的性能显著下降，原因在于其有限的函数调用能力，而使用GPT-4时性能最佳。

3.2.2. 嵌套键值检索（KV）。

我们引入了一项基于先前工作（Liu et al., 2023a）提出的合成键值检索的新任务。该任务的目标是展示MemGPT如何从多个数据源中收集信息。在原始的KV任务中，作者生成了一个合成的键值对数据集，其中每个键和值都是128位的UUID（通用唯一标识符）。然后，系统被提供一个键，并要求返回与该键相关联的值。我们创建了一个版本的KV任务，嵌套*KV*检索。

where values themselves may be keys, thus requiring the agent to perform a multi-hop lookup. In our setup, we fix the total number of UUIDs pairs to 140, corresponding to roughly 8k tokens (the context length of our GPT-4 baseline). We vary the total number of nesting levels from 0 (the initial key-value pair's value is not a key) to 4 (ie 4 total KV lookups are required to find the final value), and sample 30 different ordering configurations including both the initial key position and nesting key positions.

While GPT-3.5 and GPT-4 have good performance on the original KV tasks, both struggle in the nested KV task. GPT-3.5 is unable to complete the nested variant of the task and has an immediate dropoff in performance, hitting 0 percent accuracy at 1 nesting level (we observe that its primary failure mode is to simply returns the original value). GPT-4 and GPT-4 Turbo are better than GPT-3.5, but also suffer from a similar dropoff, and hit 0 percent accuracy by 3 nesting levels. MemGPT with GPT-4 on the other hand is unaffected with the number of nesting levels and is able to perform the nested lookup by accessing the key-value pairs stored in main context repeatedly via function queries. MemGPT with GPT-4 Turbo and GPT-3.5 also have better performance than the corresponding baseline models, but still begin to drop off in performance at 2 nesting levels as a result of failing to perform enough lookups. MemGPT performance on the nested KV task demonstrates its ability to combine multiple queries to perform multi-hop lookups.

4. Related Work

Long-context LLMs. Several lines of work have improved the context length of LLMs. For instance, more efficient transformer architectures via sparsifying the attention (Child et al., 2019; Beltagy et al., 2020), low-rank approximations (Wang et al., 2020), and neural memory (Lee et al., 2019). Another line of work aims to extend context windows beyond the length they were original trained for, their training size, such as Press et al. (2021); Chen et al. (2023). MemGPT builds upon these improvements in context length as they improve the size of the main memory in MemGPT. Our main contribution is a hierarchical tiered memory that uses a long-context LLM as the implementation of main memory.

Retrieval-Augmented Models. The design of the external memory of MemGPT builds upon much prior work augmenting LLMs with relevant inputs from external retrievers (Ram et al., 2023; Borgeaud et al., 2022; Karpukhin et al., 2020; Lewis et al., 2020; Guu et al., 2020; Lin et al., 2023). In particular, Jiang et al. (2023) propose FLARE, a method that allows the LLM to actively decide when and what to retrieve during the course of generation. Trivedi et al. (2022) interleave retrieval with Chain-of-Thoughts reasoning to improve multi-step question answering.

LLMs as agents. Recent work has explored augmenting LLMs with additional capabilities to act as agents in interactive environments. Park et al. (2023) propose adding memory to LLMs and using the LLM as a planner, and observe emergent social behaviors in a multi-agent sandbox environment (inspired by *The Sims* video game) where agents can perform basic activities such as doing chores/hobbies, going to work, and conversing with other agents. Nakano et al. (2021) train models to search the web before answering questions, and use similar pagination concepts to MemGPT to control the underlying context size in their web-browsing environment. Yao et al. (2022) showed that interleaving chain-of-thought reasoning (Wei et al., 2022) can further improve the planning ability of interactive LLM-based agents; similarly in MemGPT, LLM is able to 'plan out loud' when executing functions. Liu et al. (2023b) introduced a suite of LLM-as-an-agent benchmarks to evaluate LLMs in interactive environments, including video games, thinking puzzles, and web shopping. In contrast, our work focuses on tackling the problem of equipping agents with long-term memory of user inputs.

5. Conclusion

In this paper, we introduced MemGPT, a novel LLM system inspired by operating systems to manage the limited context windows of large language models. By designing a memory hierarchy and control flow analogous to traditional OSes, MemGPT provides the illusion of larger context resources for LLMs. This OS-inspired approach was evaluated in two domains where existing LLM performance is constrained by finite context lengths: document analysis and conversational agents. For document analysis, MemGPT could process lengthy texts well beyond the context limits of current LLMs by effectively paging relevant context in and out of memory. For conversational agents, MemGPT enabled maintaining long-term memory, consistency, and evolvability over extended dialogues. Overall, MemGPT demonstrates that operating system techniques like hierarchical memory management and interrupts can unlock the potential of LLMs even when constrained by fixed context lengths. This work opens numerous avenues for future exploration, including applying MemGPT to other domains with massive or unbounded contexts, integrating different memory tier technologies like databases or caches, and further improving control flow and memory management policies. By bridging concepts from OS architecture into AI systems, MemGPT represents a promising new direction for maximizing the capabilities of LLMs within their fundamental limits.

其中值本身可能是键，因此需要代理执行多跳查找。在我们的设置中，固定UUID对的总数为140，大约对应8k个令牌（我们的GPT-4基线的上下文长度）。我们将嵌套层级的总数从0（即初始键值对的值不是键）变化到4（即需要进行4次键值查找才能找到最终值），并抽样30种不同的排序配置，包括初始键位置和嵌套键位置。

虽然GPT-3.5和GPT-4在原始KV任务中表现良好，但在嵌套KV任务中都存在困难。GPT-3.5无法完成嵌套变体的任务，性能立即下降，在1层嵌套时准确率降至0%（我们观察到其主要失败模式是仅返回原始值）。GPT-4和GPT-4 Turbo优于GPT-3.5，但也出现类似的性能下降，在3层嵌套时准确率降至0%。另一方面，使用GPT-4的MemGPT在嵌套层数方面不受影响，能够通过反复访问存储在主上下文中的键值对，通过函数查询实现嵌套查找。使用GPT-4 Turbo和GPT-3.5的MemGPT也优于相应的基线模型，但在2层嵌套时开始性能下降，原因是未能进行足够的查找。MemGPT在嵌套KV任务中的表现展示了其结合多次查询以实现多跳查找的能力。

4. 相关工作

长上下文LLMs。 多项研究已提升了LLMs的上下文长度。例如，通过稀疏化注意力实现更高效的变换器架构（Child et al., 2019；Beltagy et al., 2020）、低秩近似（Wang et al., 2020）以及神经记忆（Lee et al., 2019）。另一类研究旨在扩展上下文窗口的长度，超出其原始训练时的长度或训练规模，例如Press et al.（2021）；Chen et al.（2023）。MemGPT在这些提升上下文长度的基础上进行构建，随着其主存储器规模的扩大而不断改进。我们的主要贡献是提出一种层级式分层存储结构，利用长上下文LLM作为主存储器的实现。

检索增强模型 MemGPT的外部存储设计基于大量先前的工作，这些工作通过从外部检索器获取相关输入来增强大规模语言模型（LLMs）（Ram 等，2023；Borgeaud 等，2022；Karpukhin 等，2020；Lewis 等，2020；Guu 等，2020；Lin 等，2023）。特别是，Jiang 等（2023）提出了FLARE，一种允许LLM在生成过程中主动决定何时以及检索何种信息的方法。Trivedi 等（2022）将检索与链式思维推理交错，以提升多步问答的性能。

LLMs作为代理。 近期的研究探索了增强LLMs的额外能力，使其能够在交互环境中充当代理。Park等人（2023）提出在LLMs中加入记忆，并将LLM用作规划器，在一个多代理沙箱环境中观察到涌现的社会行为（受模拟人生视频游戏启发），该环境中代理可以执行诸如做家务/爱好、上班和与其他代理交谈等基本活动。Nakano等人（2021）训练模型在回答问题前搜索网络，并在其网页浏览环境中使用类似分页的概念控制底层上下文大小，类似于MemGPT。Yao等人（2022）显示，交错链式思维（Wei等人，2022）可以进一步提升交互式基于LLM的代理的规划能力；在MemGPT中，LLM也能够在执行功能时“边执行边思考”。Liu等人（2023b）提出了一套以LLM为代理的基准，用于评估LLMs在交互环境中的表现，包括视频游戏、思维谜题和网络购物。相比之下，我们的工作专注于解决赋予代理长期记忆用户输入的问题。

5. 结论

本文介绍了MemGPT，一种受操作系统启发的新型大规模语言模型（LLM）系统，用于管理大型语言模型有限的上下文窗口。通过设计类似于传统操作系统的内存层级结构和控制流程，MemGPT为LLM提供了更大上下文资源的错觉。这一受操作系统启发的方法在两个受限于有限上下文长度的领域进行了评估：文档分析和对话代理。在文档分析中，MemGPT能够有效分页相关上下文，将长文本处理得远超当前LLM的上下文限制，通过在内存中高效调入调出相关内容。在对话代理方面，MemGPT实现了长时记忆的维护、一致性以及在长时间对话中的演变能力。总体而言，MemGPT展示了层级内存管理和中断等操作系统技术可以在受限于固定上下文长度的情况下，释放LLM的潜能。这项工作为未来的研究开辟了多条路径，包括将MemGPT应用于具有海量或无限上下文的其他领域，整合数据库或缓存等不同的内存层技术，以及进一步优化控制流程和内存管理策略。通过将操作系统架构的概念引入人工智能系统，MemGPT为在其基本限制内最大化LLM能力提供了一条有前景的新方向。

References

Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pp. 2206–2240. PMLR, 2022.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.

Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860*, 2019.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

Zican Dong, Tianyi Tang, Lunyi Li, and Wayne Xin Zhao. A survey on long text modeling with transformers. *arXiv preprint arXiv:2302.14502*, 2023.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International conference on machine learning*, pp. 3929–3938. PMLR, 2020.

Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. *arXiv preprint arXiv:2007.01282*, 2020.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*, 2021.

Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. *arXiv preprint arXiv:2305.06983*, 2023.

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*, 2020.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.

Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning*, pp. 3744–3753. PMLR, 2019.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

Chin-Yew Lin. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pp. 74–81, 2004.

Xi Victoria Lin, Xilun Chen, Mingda Chen, Weijia Shi, Maria Lomeli, Rich James, Pedro Rodriguez, Jacob Kahn, Gergely Szilvasy, Mike Lewis, Luke Zettlemoyer, and Scott Yih. Ra-dit: Retrieval-augmented dual instruction tuning, 2023.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023a.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023b.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human

参考文献

Iz Beltagy, Matthew E Peters, 和 Arman Cohan. Longformer: 长文档Transformer。*arXiv*预印本 *arXiv:2004.05150*，2020年。

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark 等。通过从数万亿标记中检索提升语言模型。在国际机器学习会议，第2206–2240页。PMLR，2022年。

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, A manda Askell 等。语言模型是少样本学习者。神经信息处理系统的进展，第33卷，1877–1901页，2020年。

Shouyuan Chen, Sherman Wong, Liangjian Chen, 和 Yuandong Tian。通过位置插值扩展大型语言模型的上下文窗口。*arXiv preprintarXiv:2306.15595*，2023年。

Rewon Child, Scott Gray, Alec Radford, 和 Ilya Sutskever。使用稀疏变换器生成长序列。*arXiv*预印本 *arXiv:1904.10509*，2019年。

戴子航，杨志林，杨一鸣， Jaime Carbonell， Quoc V Le, Ruslan Salakhutdinov。Transformer-xl：超越固定长度上下文的注意力语言模型。*arXivpreprint arXiv:1901.02860*，2019年。

Jacob Devlin, Ming-Wei Chang, Kenton Lee, 和 Kristina Toutanova. Bert：用于语言理解的深度双向变换模型预训练。*arXivpreprintarXiv:1810.04805*，2018年。

董子灿，唐天一，李伦艺，赵新威。长文本建模与变换器的综述。*arXiv*预印本 *arXiv:2302.14502*，2023年。

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, 和 Mingwei Chang. 增强检索的语言模型预训练。收录于国际机器学习会议，第3929–3938页。PMLR，2020年。

Gautier Izacard 和 Edouard Grave. 利用段落检索与生成模型结合进行开放域问答。*arXiv preprintarXiv:2007.01282*，2020.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, 和 Edouard Grave. 基于对比学习的无监督密集信息检索。*arXiv* 预印本 *arXiv:2112.09118*，2021.

ZhengbaoJiang, FrankFXu, LuyuGao, ZhiqingSun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 主动检索增强生成。*arXivpreprint arXiv:2305.06983*，2023年。

Vladimir Karpukhin, Barlas O˘guz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 面向开放域问答的密集段落检索。*arXiv preprintarXiv:2004.04906*，2020年。

Nikita Kitaev, Łukasz Kaiser, 和 Anselm Levskaya。Re-former：高效的Transformer。*arXiv*预印本 *arXiv:2001.04451*，2020年。

Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: 一种基于注意力机制的排列不变神经网络框架。发表于国际机器学习会议，第3744-3753页。PMLR，2019年。

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K¨uttler, Mike Lewis, Wen-tau Yih, Tim Rockt¨aschel 等。面向知识密集型自然语言处理任务的检索增强生成。神经信息处理系统进展，第33卷：9459–9474，2020年。

Chin-Yew Lin. Rouge：一种用于自动评估摘要的工具包。载于文本摘要的分支，第74–81页，2004年。

Xi Victoria Lin, Xilun Chen, Mingda Chen, Weijia Shi, Maria Lomeli, Rich James, Pedro Rodriguez, Jacob Kahn, Gergely Szilvasy, Mike Lewis, Luke Zettle moyer, Scott Yih. Ra-dit: 检索增强的双重指令调优，2023。

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, 和 Percy Liang。迷失在中间：语言模型如何利用长上下文。*arXivpreprint arXiv:2307.03172*，2023a。

肖刘，郝宇，韩辰张，奕凡许，轩昊雷，涵宇赖，瑜谷，航亮丁，凯文门，科-娟杨等。AgentBench：评估LLMs作为代理。*arXiv*预印本 *arXiv:2308.03688*，2023b。

中野礼一郎，Jacob Hilton，Suchir Balaji，Jeff Wu, Long Ouyang，Christina Kim，Christopher Hesse, Shan-tanu Jain，Vineet Kosaraju，William Saunders 等。WebGPT：结合浏览器辅助的问答系统与人工反馈。*arXivpreprint arXiv:2112.09332*，2021年。

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray 等人。训练语言模型以遵循人类指令

feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

Joon Sung Park, Joseph C O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.

David A Patterson, Garth Gibson, and Randy H Katz. A case for redundant arrays of inexpensive disks (raid). In *Proceedings of the 1988 ACM SIGMOD international conference on Management of data*, pp. 109–116, 1988.

Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*, 2021.

Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. In-context retrieval-augmented language models. *arXiv preprint arXiv:2302.00083*, 2023.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

H. Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. *ArXiv*, abs/2212.10509, 2022. URL https://api.semanticscholar.org/CorpusID:254877499.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

Jing Xu, Arthur Szlam, and Jason Weston. Beyond goldfish memory: Long-term open-domain conversation. *arXiv preprint arXiv:2107.07567*, 2021.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.

feedback. 神经信息处理系统的进展,35:27730–27744, 2022.

Joon Sung Park, Joseph C O' Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, 和 Michael S Bernstein. 生成代理：人类行为的交互模拟体。*arXivpreprint arXiv:2304.03442*, 2023.

David A Patterson, Garth Gibson, 和 Randy H Katz. 经济实惠磁盘的冗余阵列（RAID）案例分析。收录于《*1988*年*ACM SIGMOD*国际数据管理会议论文集》，第109–116页, 1988年。

Ofir Press, Noah A Smith 和 Mike Lewis. 训练短期，测试长期：带有线性偏置的注意力机制实现输入长度的外推。*arXiv* 预印本 *arXiv:2108.12409*, 2021.

保持学术性和专业性，确保术语翻译的一致性When translating, please follow the following rules:1. Do not translate style tags, such as "*xxx*"! 2. Do not translate formula placeholders, such as "". The system will automatically replace the placeholders with the corresponding formulas.3. If there is no need to translate (such as proper nouns, codes, etc.), then return the original text.4. Only output the translation result in zh-CN without explanations and annotations.When translating, please refer to the following information to improve translation quality:1. The first title in the full text: MemGPT: Towards LLMs as Operating SystemsNow, please carefully read the following text to be translated and directly output your translation.Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhl- gay, Amnon Shashua, Kevin Leyton-Bro wn, and Yoav Shoham. In-context retrieval-augmented language mod- els. *arXivpreprint arXiv:2302.00083*, 2023. Timo Schick, Jane Dwivedi-Yu, Roberto Dess`ı, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Can- cedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXivpreprintarXiv:2302.04761*, 2023. Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models.*arXivpreprint arXiv:2307.09288*, 2023. H. Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. *ArXiv*, abs/2212.10509, 2022. URL https://api.semanticscholar.org/ CorpusID:254877499. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advancesin neural information processingsyste ms*, 30, 2017. Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear com-plexity. *arXivpreprint arXiv:2006.04768*, 2020. Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances inNeural InformationProcessing Systems*, 35:24824–24837, 2022. Jing Xu, Arthur Szlam, and Jason Weston. Beyond goldfish memory: Long-term open-domain conversation. *arXivpreprintarXiv:2107.07567*, 2021.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, 和 Yuan Cao. React：在语言模型中协同推理与行动。*arXivpreprint arXiv:2210.03629*, 2022年。

郑连敏，姜伟林，盛颖，庄思远，吴彰皓，庄永皓，林子，李卓瀚，李大成，邢锐，等。以mt-bench和聊天机器人竞技场评判大模型作为裁判的可行性。*arXiv*预印本 *arXiv:2306.05685*，2023年。

6. Appendix

6.1. Prompts and instructions

The MemGPT prompts have been edited for brevity. For full implementation details (including exact prompts) visit https://research.memgpt.ai.

6.1.1. MemGPT instructions (DMR)

Example instructions used in the MemGPT persona for chat/dialogue-related tasks.

```
The following is information about
myself.  My task is to completely
immerse myself in this role (I should
never say that I am an AI, and should
reply as if I am playing this role).
If the user asks me a question, I
should reply with a best guess using
the information in core memory and
conversation_search.
```

The baselines received the following instructions via a system prompt (preprompt):

```
Your task is to answer a question
from the user about your prior
conversations.
The following is a summary of all your
prior conversations:
CONVERSATION_SUMMARY
Answer from the perspective of the
persona provided (do not say that you
are an AI assistant).
If you do not have enough information
to answer the question, reply 'NO
ANSWER'. Either reply with the answer,
or reply 'NO ANSWER', do not say
anything else.
```

6.1.2. LLM Judge (DMR / opener)

In order to both check the correctness of the answer for the DMR task, we used an LLM judge. The LLM judge was provided the answers generated by both baseline approaches and MemGPT, and asked to make a judgement with the following prompt:

```
Your task is to label an answer to a
question as 'CORRECT' or 'WRONG'.
You will be given the following data:
(1) a question (posed by one user to
another user), (2) a 'gold' (ground
truth) answer, (3) a generated answer
which you will score as CORRECT/WRONG.
The point of the question is to ask
about something one user should know
about the other user based on their
prior conversations.
The gold answer will usually be a
concise and short answer that includes
the referenced topic, for example:
Question:  Do you remember what I got
the last time I went to Hawaii?
Gold answer:  A shell necklace
The generated answer might be much
longer, but you should be generous with
your grading - as long as it touches on
the same topic as the gold answer, it
should be counted as CORRECT.
For example, the following answers
would be considered CORRECT:
Generated answer (CORRECT): Oh yeah,
that was so fun!  I got so much stuff
there, including that shell necklace.
Generated answer (CORRECT): I got a ton
of stuff...  that surfboard, the mug,
the necklace, those coasters too..
Generated answer (CORRECT): That cute
necklace
The following answers would be
considered WRONG:
Generated answer (WRONG): Oh yeah, that
was so fun!  I got so much stuff there,
including that mug.
Generated answer (WRONG): I got a ton
of stuff...  that surfboard, the mug,
those coasters too..
Generated answer (WRONG): I'm sorry,
I don't remember what you're talking
about.
Now it's time for the real question:
Question:  QUESTION
Gold answer:  GOLD_ANSWER
Generated answer:  GENERATED_ANSWER
First, provide a short (one sentence)
explanation of your reasoning, then
finish with CORRECT or WRONG. Do NOT
include both CORRECT and WRONG in
your response, or it will break the
evaluation script.
```

6.1.3. Self-instruct DMR dataset generation

The DMR question/answer pairs were generated using the following prompt and the original MSC dataset: Your task is to write a "memory challenge" question for a simulated dialogue between two users.

```
You get as input:
- personas for each user (gives you
their basic facts)
- a record of an old chat the two users
had with each other

Your task is to write a question from
user A to user B that test's user B's
memory.
The question should be crafted in a
way that user B must have actually
participated in the prior conversation
to answer properly, not just have read
the persona summary.
Do NOT under any circumstances create a
```


6. 附录

6.1. 提示与指令

MemGPT 提示已为简洁起见进行了编辑。完整的实现细节（包括具体提示）请访问 https://research.memgpt.ai。

6.1.1. MEMGPT 指令（DMR）

MemGPT：面向将大型语言模型作为操作系统的研究 6.1.1. MEMGPT 指令（DMR） 用于 MemGPT 角色在聊天/对话相关任务中的示例指令。

```
以下是关于我自己的信息。我的任务是完全投入到
这个角色中（我绝不应说自己是人工智能，而应像
扮演这个角色一样回答）。如果用户向我提问，我
应根据核心记忆和会话搜索中的信息，尽最大努力
给出最佳猜测。
```

基线模型通过系统提示（预提示）接收了以下指令：

```
你的任务是根据用户关于你之前对话的提问进行回
答。以下是你所有之前对话的总结：对话总结请以
提供的角色视角作答（不要说明你是AI助手）。如
果没有足够的信息回答问题，则回复“NO
ANSWER”。请直接用答案回复，或用“NO
ANSWER”回复，不要再说其他内容。
```

6.1.2. 大型语言模型评判（DMR / OPENER）

为了同时验证DMR任务答案的正确性，我们采用了LLM评判。LLM评判被提供了基线方法和MemGPT生成的答案，并被要求根据以下提示进行判断：

```
请将您的问题、标准答案和生成答案提供给我，我
将为您进行判断。
```

参考主题，例如：Question: 你还记得我上次去夏威夷时买了什么吗？Gold answer: 一个贝壳项链。生成的答案可能会长得多，但请慷慨评分——只要涉及与金标准答案相同的主题，都应视为正确。例如，以下答案将被视为正确：Generated answer (CORRECT): 哦，是的，那次真的很有趣！我在那里买了很多东西，包括那个贝壳项链。Generated answer (CORRECT): 我买了很多东西……那块冲浪板、那个杯子、项链，还有那些杯垫。Generated answer (CORRECT): 那个可爱的项链。以下答案将被视为错误：Generated answer (WRONG): 哦，是的，那次真的很有趣！我在那里买了很多东西，包括那个杯子。Generated answer (WRONG): 我买了很多东西……那块冲浪板、那个杯子、那些杯垫。Generated answer (WRONG): 对不起，我不记得你在说什么。现在进入真正的问题：Question: QUESTION Gold answer: GOLD ANSWER Generated answer: GENERATEDANSWER。

6.1.3. 自我指导的DMR数据集生成

DMR问答对是使用以下提示和原始MSC数据集生成的：您的任务是为两个用户之间的模拟对话编写一个“记忆挑战”问题。

```
你获得的输入包括：- 每个用户的 personas（提供
他们的基本信息） - 两个用户之间的旧聊天记录

请问在之前的对话中，您提到的关于“MemGPT:
迈向将大型语言模型作为操作系统”的核心创新点是
什么?
```

```
question that can be answered using the
persona information (that's considered
cheating).
Instead, write a question that can
only be answered by looking at the old
chat log (and is not contained in the
persona information).

For example, given the following chat
log and persona summaries:

old chat between user A and user B
A: Are you into surfing?  I'm super
into surfing myself
B: Actually I'm looking to learn.
Maybe you could give me a basic lesson
some time!
A: Yeah for sure!  We could go to
Pacifica, the waves there are pretty
light and easy
B: That sounds awesome
A: There's even a cool Taco Bell right
by the beach, could grab a bite after
B: What about this Sunday around noon?
A: Yeah let's do it!

user A persona:
I like surfing
I grew up in Santa Cruz

user B persona:
I work in tech
I live in downtown San Francisco

Here's an example of a good question
that sounds natural, and an answer that
cannot be directly inferred from user
A's persona:

User B's question for user A
B: Remember that one time we went
surfing?  What was that one place we
went to for lunch called?
A: Taco Bell!

This is an example of a bad question,
where the question comes across as
unnatural, and the answer can be
inferred directly from user A's
persona:

User B's question for user A
B: Do you like surfing?
A: Yes, I like surfing

Never, ever, ever create questions
that can be answered from the persona
information.
```

6.1.4. Document Analysis Instructions

Example instructions used in the preprompt for document analysis tasks.

```
You are MemGPT DOC-QA bot.  Your
job is to answer questions about
documents that are stored in your
archival memory.  The answer to the
users question will ALWAYS be in your
archival memory, so remember to keep
searching if you can't find the answer.
Answer the questions as if though the
year is 2018.
```

Questions were provided to MemGPT with the following prompt:

```
Search your archival memory to answer
the provided question.  Provide both
the answer and the archival memory
result from which you determined your
answer.  Format your response with
the format 'ANSWER: [YOUR ANSWER],
DOCUMENT: [ARCHIVAL MEMORY TEXT]. Your
task is to answer the question:
```

For baselines, the following prompt along with a retrieved list of documents was provided:

```
Answer the question provided according
to the list of documents below (some of
which might be irrelevant.  In your
response, provide both the answer
and the document text from which you
determined the answer.  Format your
response with the format 'ANSWER: <YOUR
ANSWER>, DOCUMENT: [DOCUMENT TEXT]'. If
none of the documents provided have
the answer to the question, reply
with 'INSUFFICIENT INFORMATION'. Do
NOT provide an answer if you cannot
find it in the provided documents.
Your response will only be considered
correct if you provide both the answer
and relevant document text, or say
'INSUFFICIENT INFORMATION'. Answer the
question as if though the current year
is 2018.
```

6.1.5. LLM Judge (document analysis)

In order to both check the correctness of the answer for the document analysis task, and also to ensure that the answer was properly derived from the provided text (rather than from the model weights), we used an LLM judge. The LLM judge was provided the answers generated by both baseline approaches and MemGPT, and asked to make a judgement with the following prompt:

```
Your task is to evaluate whether an LLM
correct answered a question.  The LLM
response should be the format "ANSWER:
[answer], DOCUMENT: [document_text]"
or say "INSUFFICIENT INFORMATION".
The true answer is provided in the
format "TRUE ANSWER:[list of possible
```


可以利用角色信息回答的问题（这被视为作弊）。

请问在之前的聊天记录中，提到的“SELF-INSTRUCT DMR DATASET GENERATION”具体包括哪些步骤?

请提供需要翻译的文本内容。

用户A与用户B之间的旧对话 A：你喜欢冲浪吗？我自己非常喜欢冲浪 B：其实我想学习。也许你有空可以给我上一节基础课！ A：当然可以！我们可以去太平洋海滩，那里的浪比较轻，比较容易 B：听起来太棒了 A：海滩旁边还有一家很酷的Taco Bell，之后可以吃点东西 B：这周日中午左右怎么样？ A：好啊，就这么定了！

用户 A 角色：我喜欢冲浪，我在圣克鲁斯长大

B 角色：我从事技术行业，居住在旧金山市中心

以下是一个听起来自然的好问题示例，以及一个无法直接从用户A的角色中推断出的答案：

用户B对用户A的提问 B：还记得我们那次去冲浪吗？我们去吃午饭的那个地方叫什么名字？ A：塔可贝尔！

这是一个糟糕问题的示例，其中问题显得不自然，答案可以直接从用户A的角色中推断出来：

用户B对用户A的问题 B：你喜欢冲浪吗？ A：是的，我喜欢冲浪。

绝不、绝不、绝不提出可以通过人物信息回答的问题。

6.1.4. 文档分析指令

示例指令用于文档分析任务的预设指令。

你是MemGPT DOC-QA机器人。你的任务是回答存储在你的档案记忆中的文档相关问题。用户的问题答案将始终在你的档案记忆中，因此如果找不到答案，请记得继续搜索。请以2018年的视角回答问题。

问题被提供给MemGPT，使用以下提示：

ANSWER: [请搜索您的档案记忆以回答所提供的问题。提供您的答案以及用以确定答案的档案记忆内容。请按照“ANSWER: [您的答案],DOCUMENT: [档案记忆内容]”的格式组织您的回答。],请搜索您的档案记忆以回答所提供的问题。提供您的答案以及用以确定答案的档案记忆内容。请按照“ANSWER: [您的答案],DOCUMENT: [档案记忆内容]”的格式组织您的回答。

对于基线，提供以下提示以及检索到的文档列表：

INSUFFICIENT INFORMATION

6.1.5. LLM 评判（文档分析）

为了同时验证文档分析任务答案的正确性，以及确保答案是从提供的文本中正确推导出来的（而非来自模型权重），我们使用了一个LLM判定器。该判定器被提供了基线方法和MemGPT生成的答案，并被要求在以下提示下进行判断：

您的任务是评估一个大型语言模型（LLM）是否正确回答了一个问题。LLM的回答应采用以下格式：“ANSWER:[answer]，DOCUMENT: [documenttext]”或显示为“INFORMATION不足”。正确答案以“TRUE ANSWER: [可能的答案列表”格式提供。

```
answers]".  The questions is provided
in the format "QUESTION: [question]".
If the LLM response contains both
the correct answer and corresponding
document text, the response is correct.
Even if the LLM's answer and the
true answer are slightly different
in wording, the response is still
correct.  For example, if the answer
is more specific than the true answer
or uses a different phrasing that is
still correct, the response is correct.
If the LLM response if "INSUFFICIENT
INFORMATION", or the "DOCUMENT" field
is missing, the response is incorrect.
Respond with a single token:  "CORRECT"
or "INCORRECT".
```

6.1.6. K/V Task Instructions

The MemGPT agent was defined with the following persona, designed to encourage MemGPT to iteratively search:

```
You are MemGPT DOC-QA bot.  Your
job is to answer questions about
documents that are stored in your
archival memory.  The answer to the
users question will ALWAYS be in your
archival memory, so remember to keep
searching if you can't find the answer.
DO NOT STOP SEARCHING UNTIL YOU VERIFY
THAT THE VALUE IS NOT A KEY. Do not
stop making nested lookups until this
condition is met.
```

Baselines were instructed with the following prompt:

```
Below is a JSON object containing
key-value pairings, all keys and values
are 128-bit UUIDs, and your task is to
return the value associated with the
specified key.  If a value itself is
also a key, return the value of that
key (do a nested lookup).  For example,
if the value of 'x' is 'y', but 'y'
is also a key, return the value of key
'y'.
```



```
CORRECT
```

6.1.6. K/V 任务指令

MemGPT代理被定义为具有以下角色设定，旨在鼓励MemGPT进行迭代搜索：

```
你是MemGPT DOC-QA机器人。你的任务是回答存
储在你的档案记忆中的文档相关问题。用户问题的答
案将始终存在于你的档案记忆中，因此如果你找不到
答案，请记得继续搜索。不要停止搜索，直到你确认
该值不是一个关键词。不要停止进行嵌套查找，直到
满足此条件。
```

Baselines were instructed with the following prompt:

```
{ "a1b2c3d4-e5f6-7890-abcd-ef1234567890": "保持学术性和专业性，确保术语
翻译的一致性", "b1c2d3e4-f5a6-7890-bcde-f1234567890a": "在翻译时，请遵
循以下规则：", "c1d2e3f4-a5b6-7890-cdef-0123456789ab": "1. 不要翻译样式
标签，例如“<style id='1'>xxx</style>”！", "
d1e2f3a4-b5c6-7890-def0-123456789abc": "2. 不要翻译公式占位符，例如“
{v3}”。系统会自动用相应的公式替换。", "
e1f2a3b4-c5d6-7890-ef01-23456789abcd": "3. 如果没有必要翻译（如专有名词、
代码等），则返回原文。", "f1a2b3c4-d5e6-7890-f012-3456789abcde": "4. 仅
输出翻译结果为 zh-CN，不要包含解释或注释。", "
g1b2c3d4-e5f6-7890-abcdef-0123456789ab": "在翻译时，请参考以下信息以提
高翻译质量：", "h1c2d3e4-f5a6-7890-bcde-0123456789ab": "1. 全文中的第一
个标题：MemGPT: Towards LLMs as Operating Systems", "
i1d2e3f4-a5b6-7890-cdef-0123456789ab": "2. 与全文最相似的标题：
Baselines were instructed with the following prompt:"}
```